Volunteer Portal API Reference

This document defines the volunteer-facing endpoints: pre-approval onboarding, the personal profile and weekly availability, assigned tasks and marking them complete, the read-only events and vet-appointment views, and account closure. It follows the same conventions established in 07-Public API Reference — see that document for base URL, standard response structure, and error code definitions, which are not repeated here. Every response uses the standard envelope: `{ success, message, data, pagination? }` on success and `{ success: false, message, error: { code, details } }` on failure. Every endpoint in this document is Volunteer-only. A volunteer is identified via `req.user.userID`; their shelter is resolved server-side and is never read from the request body. All example data in this document is fictional.

1. Overview

The volunteer portal is where a shelter volunteer sees what they have been asked to do. Registration and login are documented in 06-Auth_System; this document covers everything a volunteer does after that point. It spans six areas:

- **Onboarding before approval** — a new volunteer completes a short wizard (profile, address, government ID) while still Pending (Sections 3 and 4).
- **Profile and availability** — contact details, address, and a weekly availability grid that staff can read when assigning work (4.1–4.3).
- **Tasks** — the tasks staff have assigned to the volunteer, and marking them Completed (Section 5).
- **Events** — a read-only view of the shelter's events, with the volunteer's own assignments flagged; staff assign volunteers, volunteers do not sign themselves up (6.1).
- **Vet appointments they assist** — a read-only list of the appointments staff have assigned the volunteer to help with (6.2).
- **Closing the account** — deactivate or permanently delete, with history preserved (7.1 and Section 9).

The table below lists every endpoint a volunteer can call. Paths are relative to `/api/v1`, whose base URL is:

```
Development:  http://localhost:5000/api/v1
Production:   https://petpals-api.onrender.com/api/v1
```

The Pending column shows whether a Pending volunteer may call the route (see Section 2). All twelve endpoints are Volunteer-only; the staff endpoints that act on volunteers are summarised in Section 10, and their full behaviour remains in 11-Staff API Reference.

Endpoint	Roles	Pending	Purpose
GET /volunteers/me	Volunteer	Yes	Own profile (4.1)
PUT /volunteers/me	Volunteer	Yes	Update own profile (4.2)
PUT /volunteers/me/availability	Volunteer	No	Set weekly availability (4.3)
PATCH /volunteers/me/onboarding-step	Volunteer	Yes	Advance the onboarding wizard (4.4)
PATCH /volunteers/me/onboarding-complete	Volunteer	Yes	Finish onboarding (4.5)
GET /volunteers/me/government-id	Volunteer	Yes	Government ID status, masked (4.6)
POST /volunteers/me/government-id	Volunteer	Yes	Submit or resubmit government ID (4.7)
GET /volunteers/me/tasks	Volunteer	No	Tasks assigned to the volunteer (5.1)
PATCH /volunteers/me/tasks/:id/status	Volunteer	No	Mark a task Completed (5.2)
GET /volunteers/me/events	Volunteer	No	Shelter events, read-only (6.1)
GET /volunteers/me/appointments	Volunteer	No	Vet appointments assisted, read-only (6.2)

Endpoint	Roles	Pending	Purpose
DELETE /volunteers/me	Volunteer	No	Deactivate or delete own account (7.1)

> ***Never returned:*** *userPassword, refreshToken, the full government ID number (always masked) and stripeCustomerID are excluded from every response in this document, regardless of role — this mirrors the NF-10 data privacy requirement.*

2. Volunteer Lifecycle & Access Rules

A volunteer moves through three account states. A Pending volunteer is allowed to log in, so that onboarding can happen before approval.

State	Can log in	API access	How the volunteer gets here
Pending	Yes	Only the allowPending routes in 2.2; every other route returns 401	Registration (2.1)
Active	Yes	All volunteer routes	Approved by staff at the volunteer's shelter (2.3)
Deactivated	No — rejected at login	None; an already-issued access token stops working immediately (2.4)	Declined or deactivated by staff (2.3), or self-deactivated (7.1)

2.1 Registration and login

A volunteer registers through the standard registration endpoint (see 06-Auth_System) with role "volunteer" and a `shelterID`, which is required. A successful registration creates the account in the Pending state. Volunteers sign in on the public `/login` page, not the worker portal. Closing the account (7.1) ends on the public home page.

2.2 Pending volunteers and allowPending routes

A Pending volunteer can log in. The login and refresh responses include `accountStatus`, `onboardingComplete` and `onboardingStep`, which the frontend uses to send the volunteer to the right place (the wizard, or an awaiting-approval screen). Routes opt in to Pending access through the `authenticate.allowPending` middleware. A Pending volunteer may reach only the following routes:

Route	Section
GET /volunteers/me and PUT /volunteers/me	4.1, 4.2
GET /volunteers/me/government-id and POST /volunteers/me/government-id	4.6, 4.7
PATCH /volunteers/me/onboarding-step	4.4
PATCH /volunteers/me/onboarding-complete	4.5
POST /auth/logout	06-Auth_System

Every other route returns UNAUTHORIZED (401) for a Pending volunteer — including `PUT /volunteers/me/availability` and `DELETE /volunteers/me`, so a Pending volunteer cannot set availability or close their account until they are approved or declined.

> ***Why 401 and not 403:*** *the frontend redirects every 403 response to /forbidden. A Pending volunteer is not forbidden from the portal — they simply are not approved yet — so a 403 would send them to the wrong page. 401 keeps them on the login/approval flow. This is deliberate and should not be "corrected" to 403.*

A Pending-volunteer rejection looks like any other error response:

```
{
  "success": false,
  "message": "Your account is awaiting approval",
  "error": { "code": "UNAUTHORIZED", "details": "..." }
}
```

2.3 Approval and decline

Approval belongs to the volunteer's shelter staff, not to the volunteer: `PATCH /api/v1/volunteers/:id/status` with `{ accountStatus: "Active" }` (see 11-Staff API Reference, 8.3). Any staff member at the volunteer's shelter may do it. The rules are:

- **Approving** a Pending volunteer returns CONFLICT (409) unless onboarding is complete **and** the government ID is Verified.
- **Declining** (Pending → Deactivated) is allowed at any stage, including before onboarding is finished.
- **Deactivating** an Active volunteer (Active → Deactivated) is also done by staff.
- The government ID itself is verified by any staff member at the same shelter through `PATCH /api/v1/government-ids/:id/status` (11-Staff API Reference, 10.3).

2.4 Live account-status enforcement

Account status is checked live on every protected request, not only at login. Approval, decline and deactivation therefore take effect immediately without the volunteer logging in again: a newly approved volunteer's existing session starts working, and a deactivated volunteer's already-issued access token is rejected on its next use.

2.5 Existence masking and read-only boundaries

A volunteer must not be able to discover which tasks exist, or who else is on them. Where a resource is not theirs, the API answers exactly as if it did not exist; and several things that look like they should exist do not.

Situation	Response
A task the volunteer is not assigned to on PATCH /volunteers/me/tasks/:id/status (5.2)	404 NOT_FOUND — identical to a task that does not exist
POST or DELETE /volunteers/me/events/:id/signup (6.1)	404 NOT_FOUND — no such route; staff assign volunteers
Who else is assigned to an event (6.1)	Never returned — only the caller's own assigned flag
Staff or Admin calling a volunteer-only route	403 FORBIDDEN
A Pending volunteer calling a route that is not allowPending	401 UNAUTHORIZED

3. Onboarding Wizard

A new volunteer completes a four-step wizard while still Pending — the same wizard used by staff and vets. Registration counts as step 1 and happens before the wizard. At each step the client first saves that step's data, then calls `PATCH /volunteers/me/onboarding-step` with the step just completed. The client resumes the wizard from the `onboardingStep` returned at login.

Step	Screen	Data saved first	Then call
1	Registration	Account created at registration (not part of the wizard)	—
2	Personal	volunteerName, volunteerPhone, volunteerDOB, volunteerSex, avatarSeed via PUT /volunteers/me (4.2)	PATCH /volunteers/me/onboarding-step { step: 2 }

Step	Screen	Data saved first	Then call
3	Address	addressLine1, addressLine2, city, state, zip, country via PUT /volunteers/me (4.2)	PATCH /volunteers/me/onboarding-step { step: 3 }
4	Identity	Government ID via POST /volunteers/me/government-id (4.7)	PATCH /volunteers/me/onboarding-step { step: 4 }
5	Review	Nothing new — the volunteer reviews and submits	PATCH /volunteers/me/onboarding-complete (4.5), which sets onboardingStep to 5

`onboardingStep` only ever advances: the server sets it to the larger of its current value and step + 1, capped at 5, so going back in the wizard or repeating a call never moves the volunteer backward.

Required for completion: volunteerPhone, volunteerDOB, volunteerSex, addressLine1, city, state, zip, country, and a submitted government ID. addressLine2 is optional. Once onboarding is complete the volunteer waits for staff approval, which additionally requires the government ID to be Verified (2.3). Weekly availability (4.3) is not part of the wizard; it is set after approval.

4. Profile, Availability & Onboarding Endpoints

These endpoints let the logged-in volunteer manage their own profile, availability and onboarding. All except availability (4.3) are allowPending: they work while the volunteer is still awaiting approval. All of them identify the volunteer via `req.user.userID`.

4.1 GET /api/v1/volunteers/me

Returns the logged-in volunteer's own profile, including their shelter's contact details and decoded weekly availability.

Request

Field	Value
Method	GET
Auth required	Yes — Volunteer only
Pending allowed	Yes (allowPending)
Path/Body fields	None — volunteer identified via req.user.userID

Example request

```
GET /api/v1/volunteers/me
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Profile retrieved successfully",
  "data": {
    "userID": 21,
    "volunteerCode": "VOL-00021",
    "avatarSeed": "a1b2c3d4-...",
    "volunteerName": "Maya Patel",
    "volunteerPhone": "+15555550177",
    "volunteerDOB": "1998-07-21",
    "volunteerSex": "F",
```

```
      "addressLine1": "100 Example Street",
      "addressLine2": null,
      "city": "Athens",
      "state": "GA",
      "zip": "30601",
      "country": "USA",
      "shelterID": 3,
      "shelter": {
        "shelterName": "Athens Shelter",
        "shelterAddress": "200 Sample Avenue, Athens, GA",
        "shelterPhone": "+15555550100",
        "shelterEmail": "info@athens-shelter.example"
      },
      "availability": { "Mon": ["Morning", "Afternoon"], "Wed": ["Evening"] },
      "accountStatus": "Active",
      "onboardingComplete": true,
      "onboardingStep": 5,
      "createdAt": "2026-08-01T10:00:00Z",
      "emailVerified": true,
      "lastLoginAt": "2026-10-03T08:30:00Z",
      "user": { "userEmail": "maya.patel@example.com" }
    }
  }
```


`volunteerPhone` is E.164 and `volunteerSex` is M, F or O. `availability` is the decoded week (see 4.3 and 8.3), or `null` when the stored `volunteerSchedule` is older free text that cannot be decoded; in that case `volunteerSchedule` holds the raw text. `emailVerified` and `lastLoginAt` come from the USERS table, not the VOLUNTEER table, and `user.userEmail` is the login email. Passwords and refresh tokens are never included.

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Volunteer record does not exist	NOT_FOUND	404

4.2 PUT /api/v1/volunteers/me

Partially updates the logged-in volunteer's own profile. Only fields present in the body are changed.

Request

Field	Value
Method	PUT
Auth required	Yes — Volunteer only
Pending allowed	Yes (allowPending)
Body fields	Any subset of: avatarSeed, volunteerName (≤45 chars), volunteerPhone, volunteerDOB (date), volunteerSex (M / F / O), addressLine1, addressLine2, city, state, zip, country. volunteerPhone, volunteerDOB and volunteerSex accept null to clear them.

`volunteerPhone` is normalised to E.164 server-side; a number that cannot be parsed is rejected with VALIDATION_ERROR (422). `shelterID` and `accountStatus` are not editable here — providing either

returns 400 listing the rejected field(s), before any write occurs (shelter and status changes are staff/Admin-controlled). Availability is not edited here; use 4.3.

Example request

```
PUT /api/v1/volunteers/me
Authorization: Bearer <accessToken>
Content-Type: application/json
{
  "volunteerPhone": "(555) 555-0177",
  "volunteerSex": "F",
  "city": "Athens"
}
```

Response

```
200 OK
{
  "success": true,
  "message": "Profile updated successfully",
  "data": { "...": "same shape as GET /volunteers/me (4.1)" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Rejected field present (shelterID, accountStatus)	BAD_REQUEST	400
Invalid value (volunteerSex not M/F/O, invalid volunteerDOB, volunteerName over 45 chars, wrong type)	BAD_REQUEST	400
No updatable fields provided	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Volunteer record does not exist	NOT_FOUND	404
volunteerPhone cannot be parsed to E.164	VALIDATION_ERROR	422

4.3 PUT /api/v1/volunteers/me/availability

Replaces the volunteer's whole weekly availability. This is a full replacement, not a merge: days left out are unavailable, and an empty object clears the week.

Request

Field	Value
Method	PUT
Auth required	Yes — Volunteer only
Pending allowed	No — a Pending volunteer gets 401
Body fields	availability (object, required) — keys are days Mon–Sun; each value is a list of slots from Morning / Afternoon / Evening. {} clears the week.

Slots map to display hours Morning 8 AM–12 PM, Afternoon 12–5 PM and Evening 5–9 PM. The server stores the week compactly in `volunteerSchedule` (for example "Mon:MA;Wed:E") and always returns it decoded (8.3). An unknown day or slot name, or a day whose value is not a list, is rejected with 400 and nothing is saved.

Example request

```
PUT /api/v1/volunteers/me/availability
Authorization: Bearer <accessToken>
Content-Type: application/json
{
  "availability": {
    "Mon": ["Morning", "Afternoon"],
    "Wed": ["Evening"]
  }
}
```

Response

```
200 OK
{
  "success": true,
  "message": "Availability updated successfully",
  "data": {
    "...": "same shape as GET /volunteers/me (4.1)",
    "availability": { "Mon": ["Morning", "Afternoon"], "Wed": ["Evening"] }
  }
}
```

The response is the full profile, so the client can refresh its cached copy from this one call.

Error Cases

Scenario	Error Code	HTTP Status
availability missing or not an object; unknown day or slot; a day's value is not a list	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Volunteer record does not exist	NOT_FOUND	404

4.4 PATCH /api/v1/volunteers/me/onboarding-step

Advances the logged-in volunteer through the onboarding wizard. Called after a step's own data has been saved (see Section 3).

Request

Field	Value
Method	PATCH
Auth required	Yes — Volunteer only
Pending allowed	Yes (allowPending)
Body fields	step (int, required, 2–5) — the wizard step just completed

Advance-only: the server sets `onboardingStep = min(max(currentStep, step + 1), 5)`. A lower or equal step is ignored rather than moving the volunteer backward, and the value is capped at 5.

Example request

```
PATCH /api/v1/volunteers/me/onboarding-step
Authorization: Bearer <accessToken>
Content-Type: application/json
{ "step": 3 }
```

Response

```
200 OK
{
  "success": true,
  "message": "Onboarding step updated successfully",
  "data": { "...": "same shape as GET /volunteers/me (4.1), onboardingStep: 4"
}
}
```

Error Cases

Scenario	Error Code	HTTP Status
step missing or not an integer from 2 to 5	BAD_REQUEST	400
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Volunteer record does not exist	NOT_FOUND	404

4.5 PATCH /api/v1/volunteers/me/onboarding-complete

Marks onboarding complete — the final submit of the wizard's Review step.

Request

Field	Value
Method	PATCH
Auth required	Yes — Volunteer only
Pending allowed	Yes (allowPending)
Body fields	None

Requires volunteerPhone, volunteerDOB, volunteerSex, addressLine1, city, state, zip, country and a submitted government ID (of any verification status). Sets `onboardingComplete` to true and `onboardingStep` to 5. The account stays Pending until staff approve it, which also requires the ID to be Verified (2.3).

Example request

```
PATCH /api/v1/volunteers/me/onboarding-complete
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Onboarding completed successfully",
  "data": { "...": "same shape as GET /volunteers/me (4.1)" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
Volunteer record does not exist	NOT_FOUND	404
One or more required items are missing — the message names every missing item	CONFLICT	409

Example 409 body when only the ID is missing:

```
{
  "success": false,
  "message": "Onboarding is incomplete — missing: Government ID",
  "error": { "code": "CONFLICT", "details": "..." }
}
```

4.6 GET /api/v1/volunteers/me/government-id

Returns the logged-in volunteer's submitted government ID, with the ID number masked.

Request

Field	Value
Method	GET
Auth required	Yes — Volunteer only
Pending allowed	Yes (allowPending)
Path/Body fields	None — volunteer identified via req.user.userID

Response

```
200 OK
{
  "success": true,
  "message": "Government ID retrieved successfully",
  "data": {
    "governmentIDID": 14,
    "userID": 21,
    "userType": "Volunteer",
    "idType": "Passport",
    "idNumber": "*****4567",
    "verificationStatus": "Pending",
    "documentURL": "..."
  }
}
```

Response masking: `idNumber` is masked (e.g. *****4567) on this endpoint and on the POST response — the full number is never returned to the volunteer, and staff see it masked on the volunteer detail endpoint (Section 10).

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401

Scenario	Error Code	HTTP Status
Role not permitted	FORBIDDEN	403
No government ID has been submitted yet	NOT_FOUND	404

4.7 POST /api/v1/volunteers/me/government-id

Submits a government ID for identity verification. The file is stored in the private government-ids Storage bucket; only metadata is persisted in the GOVERNMENT_ID table.

Request

Field	Value
Method	POST
Auth required	Yes — Volunteer only
Pending allowed	Yes (allowPending)
Content-Type	multipart/form-data
Body fields	idType (string, required, ≤45 chars), idNumber (string, required, ≤45 chars), file (JPEG/PNG/WebP/HEIC/PDF, ≤5 MB, required)

Example request

```
POST /api/v1/volunteers/me/government-id
Authorization: Bearer <accessToken>
Content-Type: multipart/form-data
idType=Passport
idNumber=P1234567
file=@passport-photo-page.jpg
```

Response

```
201 Created
{
  "success": true,
  "message": "Government ID submitted successfully",
  "data": {
    "governmentIDID": 14,
    "userID": 21,
    "userType": "Volunteer",
    "idType": "Passport",
    "idNumber": "*****4567",
    "verificationStatus": "Pending",
    "documentURL": "..."
  }
}
```

One per volunteer: a second submission while the existing ID is Pending or Verified returns CONFLICT (409). A **Rejected** ID can be resubmitted: the existing row is overwritten in place and reset to Pending.

Error Cases

Scenario	Error Code	HTTP Status
Missing idType, idNumber or file; idType/idNumber too long	BAD_REQUEST	400
File exceeds 5 MB or is not an accepted type	BAD_REQUEST	400

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
A government ID already exists and is not Rejected	CONFLICT	409

5. Tasks

Staff create tasks and assign them to volunteers (11-Staff API Reference, Section 6). A volunteer sees only the tasks they are assigned to, and can move a task from In_progress to Completed. Cancelling is a staff action.

5.1 GET /api/v1/volunteers/me/tasks

Returns the tasks the logged-in volunteer is assigned to, paginated.

Request

Field	Value
Method	GET
Auth required	Yes — Volunteer only
Pending allowed	No
Query params	taskStatus (optional — In_progress / Completed / Cancelled), upcoming (optional — strictly "true" or "false"), page (int, optional, default 1), limit (int, optional, default 20, max 100)

`upcoming=true` returns tasks due now or later, plus tasks with no due date, soonest first. `upcoming=false` returns tasks whose due date has passed, most recent first. With `upcoming` omitted, all tasks are returned, soonest due first. Any value other than the exact strings "true" or "false" is rejected rather than ignored.

Example request

```
GET /api/v1/volunteers/me/tasks?
taskStatus=In_progress&upcoming=true&page=1&limit=20
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Tasks retrieved successfully",
  "data": [
    {
      "taskID": 14,
      "taskName": "Feeding",
      "taskDesc": "Evening feeding round, kennels 1–6",
      "taskDate": "2026-09-25T08:00:00Z",
      "taskDue": "2026-10-06T17:00:00Z",
      "taskStatus": "In_progress",
      "status": "In_progress",
      "staffName": "Jordan Lee",
      "volunteers": [ { "volunteerID": 21, "volunteerName": "Maya Patel" } ]
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
```

```
}
```

`taskName` is one of Animal_Care, Vet_Assistance, Cleaning, Feeding, Events, Admin or Other. `taskStatus` is the stored value; `status` is the displayed value, which adds "Overdue" for an In_progress task whose due date has passed (8.1). `volunteers` lists everyone assigned to the task.

Error Cases

Scenario	Error Code	HTTP Status
Invalid taskStatus or upcoming value; invalid page or limit	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

5.2 PATCH /api/v1/volunteers/me/tasks/:id/status

Marks one of the volunteer's own tasks Completed.

Request

Field	Value
Method	PATCH
Auth required	Yes — Volunteer only
Pending allowed	No
Path params	id (int, required) — taskID
Body fields	taskStatus (string, required — "Completed")

"Completed" is the only transition a volunteer can make; cancelling a task is staff's (11-Staff API Reference, 6.5). A task the volunteer is not assigned to answers 404, exactly as a task that does not exist (2.5). Only an In_progress task can be completed, and that includes one displayed as Overdue.

Example request

```
PATCH /api/v1/volunteers/me/tasks/14/status
Authorization: Bearer <accessToken>
Content-Type: application/json
{ "taskStatus": "Completed" }
```

Response

```
200 OK
{
  "success": true,
  "message": "Task updated successfully",
  "data": { "...": "same shape as the GET /volunteers/me/tasks list item
(5.1)" }
}
```

Error Cases

Scenario	Error Code	HTTP Status
Missing or invalid id; taskStatus missing or not "Completed"	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

Scenario	Error Code	HTTP Status
Task does not exist, or the volunteer is not assigned to it	NOT_FOUND	404
Task is not In_progress (already Completed or Cancelled)	CONFLICT	409

6. Events & Vet Appointments

Both views are read-only. Staff decide who is assigned; the volunteer sees the result.

6.1 GET /api/v1/volunteers/me/events

Returns every event at the volunteer's shelter, with a flag showing which ones the volunteer has been assigned to. The list item has the same shape as the public `GET /events` (07-Public API Reference) plus `assigned`.

Request

Field	Value
Method	GET
Auth required	Yes — Volunteer only
Pending allowed	No
Query params	upcoming (optional — strictly "true" or "false"), assigned (optional — strictly "true"), dateTo (optional ISO date-time, inclusive cap on eventDate), page (int, optional, default 1), limit (int, optional, default 20, max 100)

`upcoming=true` returns events that have not started yet, soonest first. `upcoming=false` returns events that have already started, most recent first. `assigned=true` returns only the events the volunteer is on. Any invalid value is rejected rather than ignored.

Example request

```
GET /api/v1/volunteers/me/events?upcoming=true&assigned=true&page=1&limit=20
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Events retrieved successfully",
  "data": [
    {
      "eventID": 7,
      "eventName": "Fall Adoption Day",
      "eventDate": "2026-10-12T11:00:00Z",
      "eventDesc": "Meet adoptable pets and shelter staff.",
      "eventCategory": "Adoption_Event",
      "shelter": { "shelterID": 3, "shelterName": "Athens Shelter" },
      "assigned": true
    }
  ],
  "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
}
```

> ***No sign-up route:*** *volunteers do not sign themselves up for events — staff assign them with volunteerIDs on POST/PUT /events (Section 10). `assigned` reflects only the caller's own assignment and never reveals who else is on the event. POST and DELETE /volunteers/me/events/:id/signup return 404.*

Error Cases

Scenario	Error Code	HTTP Status
Invalid upcoming, assigned or dateTo value; invalid page or limit	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

6.2 GET /api/v1/volunteers/me/appointments

Returns the vet appointments staff have assigned the logged-in volunteer to assist (the appointment's `volunteerID`), paginated. Read-only.

Request

Field	Value
Method	GET
Auth required	Yes — Volunteer only
Pending allowed	No
Query params	upcoming (optional — strictly "true" or "false", default "false"), page (int, optional, default 1), limit (int, optional, default 20, max 100)

`upcoming=true` returns Scheduled appointments dated in the future, soonest first. `upcoming=false` returns past-dated or Cancelled appointments, most recent first. Any value other than the exact strings "true" or "false" is rejected rather than ignored.

Example request

```
GET /api/v1/volunteers/me/appointments?upcoming=true&page=1&limit=20
Authorization: Bearer <accessToken>
```

Response

```
200 OK
{
  "success": true,
  "message": "Appointments retrieved successfully",
  "data": [
    {
      "appointmentID": 123,
      "appointmentDate": "2026-10-05T10:00:00Z",
      "appointmentReason": "Annual checkup",
      "status": "Scheduled",
      "pet": {
        "petID": 9,
        "petCode": "PE000009",
        "petName": "Buddy",
        "petPhoto": "...",
        "breedName": "Beagle",
        "speciesName": "Dog"
      },
      "vetName": "Dr. Jay Asarathi"
```

```
      }
    ],
    "pagination": { "page": 1, "limit": 20, "total": 1, "totalPages": 1 }
  }
```

The list item has the same shape and status labels as the vet's `GET /vets/me/appointments` (12-Veterinarian API Reference, 5.1). `status` is the displayed status (8.2). `vetName` is null once the vet has deleted their account.

Error Cases

Scenario	Error Code	HTTP Status
upcoming is not "true"/"false"; invalid page or limit	BAD_REQUEST	400
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403

7. Closing the Account

7.1 DELETE /api/v1/volunteers/me

Closes the volunteer's own account, either by deactivating it (data retained) or permanently deleting it. See Section 9 for exactly what each mode keeps.

Request

Field	Value
Method	DELETE
Auth required	Yes — Volunteer only
Pending allowed	No — a Pending volunteer gets 401
Body fields	mode (string, required — "deactivate" or "delete")

`deactivate` keeps the Volunteer row and sets `accountStatus` to Deactivated; the refresh token is cleared. `delete` removes the Volunteer and Users rows, the volunteer's past task and event assignment rows, and the government ID together with its stored Storage file. Appointments the volunteer assisted stay, with their volunteer link set to null. Either mode also removes the volunteer from events that have not happened yet.

Both modes are blocked while the volunteer is **assisting an upcoming Scheduled appointment** or **holding an In_progress task**: a volunteer cannot walk away from work staff are still counting on. The 409 names whichever apply in `error.details.blockers`.

Example request

```
DELETE /api/v1/volunteers/me
Authorization: Bearer <accessToken>
Content-Type: application/json
{ "mode": "deactivate" }
```

Response

```
200 OK
{
  "success": true,
  "message": "Account deactivated",
  "data": null
}
```

The message is "Account deleted" when mode is delete. After a successful close the client clears the session and ends on the public home page.

Error Cases

Scenario	Error Code	HTTP Status
No/invalid token, or the account is Pending or Deactivated	UNAUTHORIZED	401
Role not permitted	FORBIDDEN	403
mode missing or not "deactivate"/"delete"	VALIDATION_ERROR	422
The volunteer is assisting an upcoming Scheduled appointment and/or holds an In_progress task (both modes)	CONFLICT	409

Example 409 body when both blockers apply:

```
{
  "success": false,
  "message": "You have upcoming appointments and open tasks — ask your
shelter's staff to reassign them before closing your account",
  "error": {
    "code": "CONFLICT",
    "details": { "blockers": ["appointments", "tasks"] }
  }
}
```

`blockers` contains only the items that apply (`"appointments"`, `"tasks"`, or both), and the message names the same items. Staff clear them by reassigning: `PATCH /appointments/:id` with `volunteerID` (11-Staff API Reference, 4.4) and `PATCH /tasks/:id` with `volunteerIDs` (11-Staff API Reference, 6.4).

8. Status & Derived Values

Several values the API returns are computed or stored in a compact form. This section defines them so clients do not re-derive them.

8.1 Task status: displayed vs stored

A task has a stored `taskStatus` (In_progress, Completed or Cancelled). The `status` field returned in the task list is the **displayed** status, derived from the stored value and the due date.

Stored taskStatus	taskDue	Displayed status
In_progress	In the future, or no due date	In_progress
In_progress	In the past	Overdue
Completed	Any	Completed
Cancelled	Any	Cancelled

- **Filters follow different fields.** `taskStatus=` filters on the stored value, so Overdue tasks are returned by `taskStatus=In_progress`; `upcoming` filters on `taskDue` alone (5.1).
- **Overdue tasks can still be completed.** PATCH /volunteers/me/tasks/:id/status checks the stored value, so an Overdue task is simply In_progress (5.2).

8.2 Appointment status: displayed vs stored

An appointment has a stored `appointmentStatus` (Scheduled, Completed or Cancelled). The `status` field in the volunteer's list is the **displayed** status, derived from the stored value and the appointment date — the same labels the vet sees.

Stored appointmentStatus	appointmentDate	Displayed status
Scheduled	In the future	Scheduled
Scheduled	In the past	Completed
Completed	Any	Completed
Cancelled	Any	Cancelled

Display is a convenience: a past-dated Scheduled appointment reads Completed even if no vet has completed it yet. `upcoming=true` in 6.2 returns Scheduled appointments in the future; `upcoming=false` returns past-dated or Cancelled ones.

8.3 Availability encoding

Availability is a week grid of days and slots. The API accepts and returns it as readable names; the database holds one compact string in `volunteerSchedule`.

Slot name	Stored letter	Display hours	Notes
Morning	M	8 AM–12 PM	Days are Mon, Tue, Wed, Thu, Fri, Sat, Sun
Afternoon	A	12–5 PM	A day not listed means unavailable all day
Evening	E	5–9 PM	{} clears the whole week

- **Example.** `{ "Mon": ["Morning","Afternoon"], "Wed": ["Evening"] }` is stored as `Mon:MA;Wed:E` and returned in the first form.
- **Older free text.** A `volunteerSchedule` written before this grid existed may be free text. It cannot be decoded, so `availability` is null and `volunteerSchedule` carries the raw text (4.1). The next successful PUT (4.3) replaces it.
- **Staff view.** GET /volunteers/:id returns the same decoded `availability` (Section 10).

9. Data Retention on Account Deletion

A volunteer can leave in two ways (7.1). Deactivating keeps everything; deleting removes the volunteer's identity and their own assignment history, but deliberately preserves the appointments they assisted.

Data	On deactivate	On delete
Volunteer row	Kept — accountStatus set to Deactivated	Removed
Users row (login identity)	Kept	Removed
Refresh token	Cleared (forces logout)	Gone with the Users row
Government ID row and its Storage file	Kept	Both removed
Assignments to events that have not happened yet	Removed	Removed
Past event assignments	Kept	Removed
Past task assignments (Completed or Cancelled)	Kept	Removed
Appointment.volunteerID (appointments assisted)	Unchanged	Set to NULL

- **Nothing open is orphaned.** Both modes are refused while the volunteer holds an In_progress task or is assisting an upcoming Scheduled appointment (7.1), so deletion never leaves staff-assigned work without a volunteer. Only past appointments end up with a null `volunteerID`.

- **Future events are cleared either way.** A deactivated or deleted volunteer no longer appears on events that have not happened yet.
- **Appointments survive.** The vet, pet and notes on an appointment are untouched when the assisting volunteer leaves.

10. Related Staff Endpoints

These are the staff-side endpoints that act on volunteers. They are specified in 11-Staff API Reference; this table records only how they interact with the volunteer behaviour above.

Endpoint	Reference	What it does for volunteers
GET /volunteers	11 — 8.1	Staff roster for the shelter. Filter by accountStatus, search by name. Each row includes onboardingComplete and governmentIdStatus so staff can see who is ready to approve.
GET /volunteers/:id	11 — 8.2	Volunteer detail, including decoded availability (8.3). The government ID number is masked.
PATCH /volunteers/:id/status	11 — 8.3	Approve (Pending → Active) or decline (Pending → Deactivated). Approval needs onboarding complete and a Verified government ID, otherwise 409 (2.3).
PATCH /government-ids/:id/status	11 — 10.3	Any staff member at the volunteer's shelter verifies or rejects the volunteer's government ID. A Rejected ID can be resubmitted (4.7).
POST /events, PUT /events/:id	11 — 7.1, 7.2	volunteerIDs assigns volunteers to an event. They then see it with assigned: true (6.1).
GET /events/:id/volunteers	11 — 7.4	Staff-only list of the volunteers assigned to an event.
POST /tasks	11 — 6.2	volunteerIDs must be Active volunteers at the task's shelter. The task then appears in the volunteer's task list (5.1).

> ***Note for 11-Staff API Reference:*** *sections 8.1–8.3 do not yet list the onboardingComplete and governmentIdStatus roster fields, the decoded availability and masked idNumber on the detail endpoint, or the 409 for approving a volunteer whose onboarding is incomplete or whose government ID is not Verified. Those rules are specified here (2.3, Section 10) and the Staff reference should be updated to match.*

11. End-to-End Flow

One volunteer, from sign-up to a visible event assignment. Each step names the endpoint that performs it.

Flow — Volunteer sign-up to first assignments

1. Sign up: POST /auth/register with role "volunteer" and a shelterID. The account is created Pending.
2. Log in on the public /login. Pending is allowed; the response carries accountStatus Pending, onboardingComplete false and the current onboardingStep. A non-allowPending route such as GET /volunteers/me/tasks returns 401.
3. Wizard, steps 2 and 3: PUT /volunteers/me with personal data, then PATCH /volunteers/me/onboarding-step { step: 2 }; PUT /volunteers/me with address data, then { step: 3 }.
4. Identity, step 4: POST /volunteers/me/government-id (multipart), then PATCH /volunteers/me/onboarding-step { step: 4 }.
5. Review: PATCH /volunteers/me/onboarding-complete. onboardingComplete becomes true and onboardingStep 5, but the account stays Pending.

6. Staff at the shelter see the volunteer in GET /volunteers with onboardingComplete true and the ID Pending. Approving now returns 409 until the ID is Verified. Any staff member verifies the ID with PATCH /government-ids/:id/status, then approves with PATCH /volunteers/:id/status { accountStatus: "Active" }.
7. The volunteer's existing session starts working at once (2.4). They set availability with PUT /volunteers/me/availability.
8. Staff create a task with POST /tasks and the volunteer's ID in volunteerIDs. It shows in GET /volunteers/me/tasks; the volunteer completes it with PATCH /volunteers/me/tasks/:id/status { taskStatus: "Completed" }, and the task now reads Completed.
9. Staff assign the volunteer to an event with volunteerIDs on POST /events or PUT /events/:id. GET /volunteers/me/events?assigned=true now returns it with assigned: true. There is nothing for the volunteer to sign up for.